人脸追踪

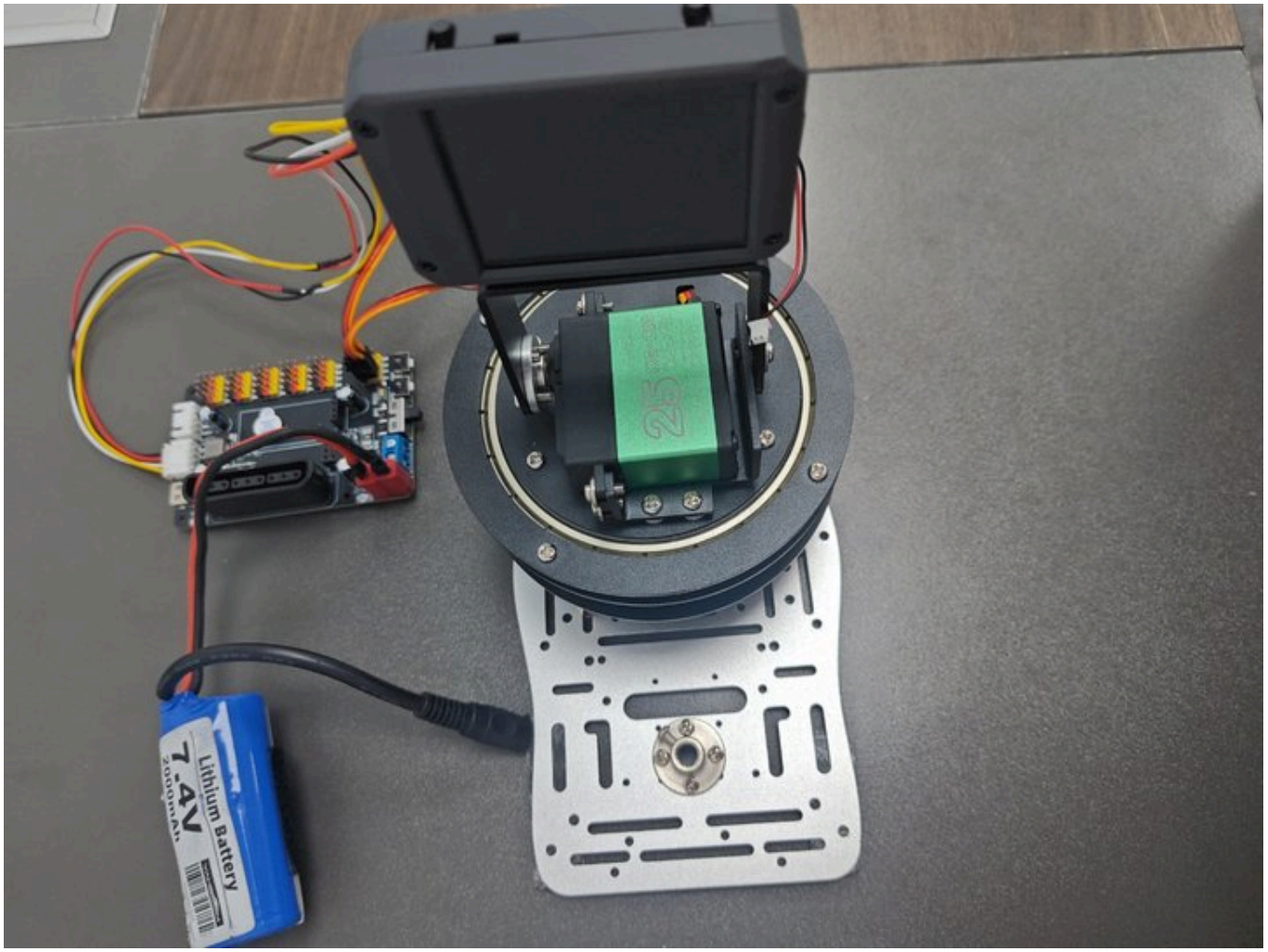
1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

2.器材清单

- k230 x1
- 24路舵机驱动板 x1
- 7.4v电池 x1 购买
- 铝合金板 x1 购买

3.摆放位置

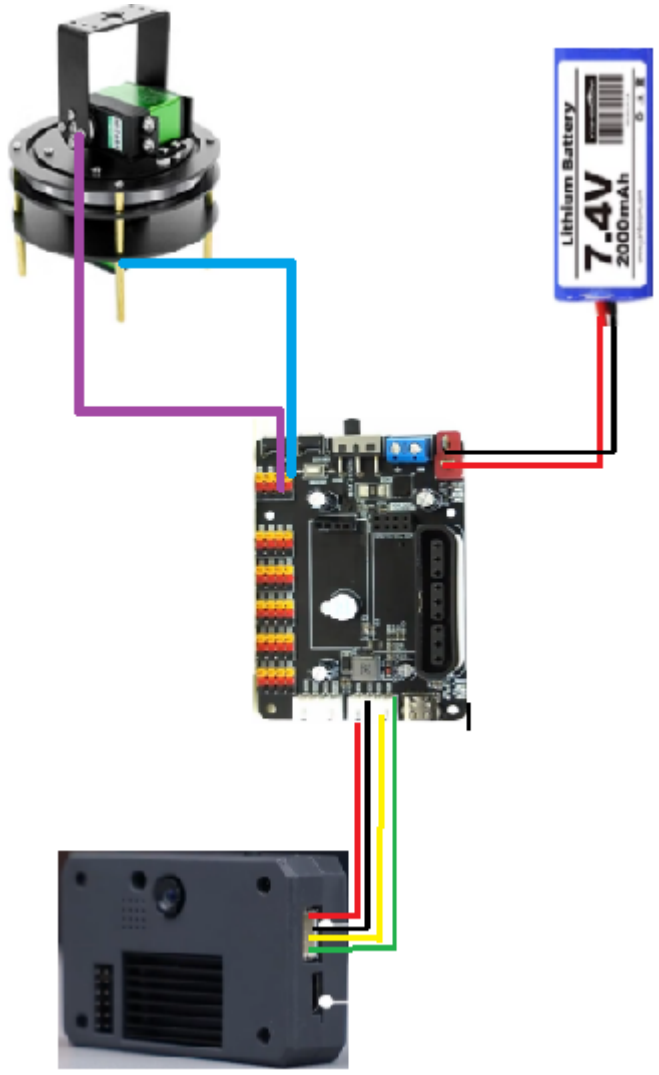


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行选择能固定的底板，底盘可以胶给粘住以免云台动的幅度大导致偏移，k230固定在二维云台上面,用电池的时候如果云台转动幅度过大可以适度减小pid参数。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230关键代码解析

- **导入关键依赖**

```
from libs.PipeLine import PipeLine, ScopedTiming
# Pipeline是集成了摄像头、LCD屏幕显示等多媒体功能的模块，大大简化了调用摄像头和显示相关的操作
# Pipeline is a module that integrates multimedia functions like camera and LCD display,
greatly simplifying camera and display operations
•
# ScopedTiming是一个代码执行计时器，使用方法可以参考例程代码
# ScopedTiming is a code execution timer, refer to example code for usage
•
from libs.AIBase import AIBase
from libs.AI2D import Ai2d
# AIBase和AI2D用来处理AI底层的逻辑
# AIBase is the base class for all AI functionality implementations
# AI2D用于处理图像
# AI2D is used for image processing
•
import os
import ujson
# os和ujson分别提供系统相关操作和JSON数据相关操作，不是每个例程都能用上
# os and ujson provide system operations and JSON data operations respectively, not required
for every example
•
from media.media import *
from time import *
import nncase_runtime as nn
# nncase是K230进行AI推理的核心模块，提供了便捷的方法供用户调用K230的KPU
# nncase is the core module for AI inference on K230, providing convenient methods for users
to call K230's KPU
•
import ulab.numpy as np
# ulab.numpy是从python的numpy中移植而来，用于进行一些AI运算中必要的矩阵操作
# ulab.numpy is ported from Python's numpy, used for necessary matrix operations in AI
computations
•
import time
import utime
import image
import random
import gc
import sys
import aidemo
# aidemo也是核心模块之一，K230固件中预制了非常多的AI玩法
# aidemo is another core module, K230 firmware includes many pre-configured AI applications
# 通过aidemo模块可以快速简单的调用这些玩法中的复杂方法
# Complex methods in these applications can be easily called through the aidemo module
•
import _thread
# _thread是线程模块，前面章节中我们有做过详细讲解，这里就不再赘述
```

```
# _thread is the threading module, as detailed in previous chapters, no need to elaborate
further
```

此代码是为了获取矩形的四个点坐标，通过找黑框的外矩形和内矩形然后确定黑框中心矩形，依次来获得中心矩形的四个顶点坐标。

- **自定义人脸检测类**

```
# 自定义人脸检测类，继承自AIBase基类
class FaceDetectionApp(AIBase):
    def __init__(self, kmodel_path, model_input_size, anchors, confidence_threshold=0.5,
nms_threshold=0.2, rgb888p_size=[224,224], display_size=[1920,1080], debug_mode=0):
        # 调用基类的构造函数 / Call parent class constructor
        super().__init__(kmodel_path, model_input_size, rgb888p_size, debug_mode)

        # 模型文件路径 / Path to the model file
        self.kmodel_path = kmodel_path

        # 模型输入分辨率 / Model input resolution
        self.model_input_size = model_input_size

        # 置信度阈值：检测结果的最小置信度要求 / Confidence threshold: minimum confidence
requirement for detection results
        self.confidence_threshold = confidence_threshold

        # NMS阈值：非极大值抑制的阈值 / NMS threshold: threshold for Non-Maximum Suppression
        self.nms_threshold = nms_threshold

        # 锚点数据：用于目标检测的预定义框 / Anchor data: predefined boxes for object detection
        self.anchors = anchors

        # sensor给到AI的图像分辨率，宽度16对齐 / Image resolution from sensor to AI, width
aligned to 16
        self.rgb888p_size = [ALIGN_UP(rgb888p_size[0], 16), rgb888p_size[1]]

        # 显示分辨率，宽度16对齐 / Display resolution, width aligned to 16
        self.display_size = [ALIGN_UP(display_size[0], 16), display_size[1]]

        # 调试模式标志 / Debug mode flag
        self.debug_mode = debug_mode

        # 实例化AI2D对象用于图像预处理 / Initialize AI2D object for image preprocessing
        self.ai2d = Ai2d(debug_mode)

        # 设置AI2D的输入输出格式 / Set AI2D input/output format
        self.ai2d.set_ai2d_dtype(nn.ai2d_format.NCHW_FMT, nn.ai2d_format.NCHW_FMT, np.uint8,
np.uint8)
•
    def config_preprocess(self, input_image_size=None):
        with ScopedTiming("set preprocess config", self.debug_mode > 0):
            # 获取AI2D输入尺寸 / Get AI2D input size
            ai2d_input_size = input_image_size if input_image_size else self.rgb888p_size
```

```
            # 获取padding参数 / Get padding parameters
            top, bottom, left, right = self.get_padding_param()

            # 设置padding: [上,下,左,右], 填充值[104,117,123] / Set padding:
[top,bottom,left,right], padding value[104,117,123]
            self.ai2d.pad([0, 0, 0, 0, top, bottom, left, right], 0, [104, 117, 123])

            # 设置resize方法：双线性插值 / Set resize method: bilinear interpolation
            self.ai2d.resize(nn.interp_method.tf_bilinear, nn.interp_mode.half_pixel)

            # 构建预处理流程 / Build preprocessing pipeline
            self.ai2d.build([1,3,ai2d_input_size[1],ai2d_input_size[0]],
                            [1,3,self.model_input_size[1],self.model_input_size[0]])
•
    def postprocess(self, results):
        with ScopedTiming("postprocess", self.debug_mode > 0):
            # 调用aidemo库进行人脸检测后处理 / Call aidemo library for face detection post-
processing
            post_ret = aidemo.face_det_post_process(self.confidence_threshold,
                                                    self.nms_threshold,
                                                    self.model_input_size[1],
                                                    self.anchors,
                                                    self.rgb888p_size,
                                                    results)
            return post_ret[0] if post_ret else post_ret
•

    def draw_result(self, pl, dets):

        with ScopedTiming("display_draw", self.debug_mode > 0):
            if dets:
                # 清除上一帧的OSD绘制 / Clear previous frame's OSD drawing
                pl.osd_img.clear()
                global last_cx
                global last_cy
                for det in dets:
                    # 转换检测框坐标到显示分辨率 / Convert detection box coordinates to display
resolution
                    x, y, w, h = map(lambda x: int(round(x, 0)), det[:4])
                   # 绘制黄色检测框 / Draw yellow detection box
                    pl.osd_img.draw_rectangle(x, y, w, h, color=(255, 255, 0, 255),
thickness=2)
                    return x , y, 1
                return last_cx, last_cy, 0
            return last_cx, last_cy, 0
•
    def get_padding_param(self):
        # 计算模型输入和实际图像的缩放比例 / Calculate scaling ratio between model input and
actual image
        dst_w = self.model_input_size[0]
        dst_h = self.model_input_size[1]
        ratio_w = dst_w / self.rgb888p_size[0]
```

```
        ratio_h = dst_h / self.rgb888p_size[1]
        ratio = min(ratio_w, ratio_h)

        # 计算缩放后的新尺寸 / Calculate new dimensions after scaling
        new_w = int(ratio * self.rgb888p_size[0])
        new_h = int(ratio * self.rgb888p_size[1])

        # 计算padding值 / Calculate padding values
        dw = (dst_w - new_w) / 2
        dh = (dst_h - new_h) / 2

        # 返回padding参数 / Return padding parameters
        return (int(round(0)),
                int(round(dh * 2 + 0.1)),
                int(round(0)),
                int(round(dw * 2 - 0.1)))
```

- **执行人脸检测**

```
def exce_demo(pl):
    # 声明全局变量face_det / Declare global variable face_det
    global face_det

    # 获取显示相关参数 / Get display-related parameters
    display_mode = pl.display_mode      # 显示模式(如lcd) / Display mode (e.g., lcd)
    rgb888p_size = pl.rgb888p_size     # 原始图像分辨率 / Original image resolution
    display_size = pl.display_size      # 显示分辨率 / Display resolution

    # 设置人脸检测模型路径 / Set face detection model path
    kmodel_path = "/sdcard/kmodel/face_detection_320.kmodel"

    # 设置模型参数 / Set model parameters
    confidence_threshold = 0.5    # 置信度阈值 / Confidence threshold
    nms_threshold = 0.2           # 非极大值抑制阈值 / Non-maximum suppression threshold
    anchor_len = 4200             # 锚框数量 / Number of anchor boxes
    det_dim = 4                   # 检测维度(x,y,w,h) / Detection dimensions (x,y,w,h)

    # 加载锚框数据 / Load anchor box data
    anchors_path = "/sdcard/utils/prior_data_320.bin"
    anchors = np.fromfile(anchors_path, dtype=np.float)
    anchors = anchors.reshape((anchor_len, det_dim))

    try:
        # 初始化人脸检测应用实例 / Initialize face detection application instance
        face_det = FaceDetectionApp(kmodel_path,
                                    model_input_size=[320, 320],
                                    anchors=anchors,
                                    confidence_threshold=confidence_threshold,
                                    nms_threshold=nms_threshold,
                                    rgb888p_size=rgb888p_size,
                                    display_size=display_size,
```

```
                                debug_mode=0)

        # 配置图像预处理 / Configure image preprocessing
        face_det.config_preprocess()

        # 主循环 / Main loop
        while True:
            with ScopedTiming("total",0):    # 计时器 / Timer
                img = pl.get_frame()          # 获取摄像头帧图像 / Get camera frame
                res = face_det.run(img)       # 执行人脸检测 / Run face detection
                face_det.draw_result(pl, res) # 绘制检测结果 / Draw detection results
                pl.show_image()               # 显示处理后的图像 / Display processed image
                x,y,ok= face_det.draw_result(pl, res)
                cx_h,cx_l,cx_z,cy_h,cy_l,cy_z = conversion_postion(x,y)
                uart.write(bytearray([0x55,0xaa,0x00,int(cx_h),int(cx_l),int(cx_z),0xfa]))
                uart.write(bytearray([0x55,0xaa,0xff,int(cy_h),int(cy_l),int(cy_z),0xfa]))
                gc.collect()                  # 垃圾回收 / Garbage collection
                time.sleep_us(10)             # 短暂延时 / Brief delay

    except Exception as e:
        print("人脸检测功能退出")           # 异常退出提示 / Exception exit prompt
    finally:
        face_det.deinit()                   # 释放资源 / Release resources
```

- **分割坐标函数**

```
def conversion_postion(x,y):
    if x > 510:
       x_h = 255
       x_l = 255
       x_z = x-255-255
    elif 510>=x>=255:
       x_h = 255
       x_l = x-255
       x_z = 0
    else:
       x_h = x
       x_l = 0
       x_z = 0
    if y > 510:
       y_h = 255
       y_l = 255
       y_z = y-255-255
    elif 510>=y>=255:
       y_h = 255
       y_l = y-255
       y_z = 0
    else:
       y_h = y
       y_l = 0
       y_z = 0
    return x_h,x_l,x_z,y_h,y_l,y_z
```

因为给stm32只是传输8位2进制数，所以最高传输数据为255，而我们K230的采用的是640x480因此需要对其坐标进行分割，将640拆成3帧数据进行传输。

6.stm32关键代码解析

- ## control.c

```
uint8_t get_point2_p()    //处理坐标函数
{
    uint8_t lable, load_1, load_2 ,load_3 ,ret=0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2,&load_3) == 0){
     if(lable == 0x00){
            rec_step_p[0][0] = load_1 + load_2 + load_3 ;
//          ret=1;

        } else if(lable == 0xff){

            rec_step_p[0][1] = load_1 + load_2 + load_3 ;
//          ret=1;

        }

    return ret;
}
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起这样形成正确的坐标，将坐标赋值给二维数组。

```
    get_point2_p();

    set_target(rec_step_p[0][0],rec_step_p[0][1]);
```

此代码是在上面获得人脸坐标的基础上将坐标作为pid目标值，然后控制舵机运动。

- ## bsp_servo.c

```
ServTypdef Xserv = {
    .pwm = 2000,
    .MAXPWM = 2500,
    .MINPWM = 500,
    .MIDPWM = 1400
};

ServTypdef Yserv = {
    .pwm = 1000,
    .MAXPWM = 2300,
    .MINPWM = 500,
```

```
    .MIDPWM = 1000
};
```

此代码是设置舵机启动的起始位置，所以需要修改这个来拟合云台运动的顶点，我将云台开机初始位置设置在右上角，然后云台可以通过k230的坐标变化来按比例进行调整，如果在调整云台位置和细调位置的时候就需要调这个位置参数了。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }

    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                          //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- **pid.c**

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
```

```
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
             + PID->Kd * (PID->Err - PID->LastErr);                      /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出，如果想微调代码想达到更好的效果可以微调主函数的pid参数。

- **bsp_usart.c**

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {            //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){      //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){      //帧
尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
```

```
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

7.效果描述

k230代码定义人脸类型，当通过特征识别到人脸的时候，k230将人脸坐标传输给二维云台，然后二维云台跟着人脸运动，从而达到追踪人脸的效果。